CSI500 Stock Selection Competition Report

Xinpeng Cheng

May 2026

1 Introduction

This project studies a machine-learning-based quantitative stock selection problem. The goal is to construct a long-only portfolio of CSI500 constituents that can outperform the CSI500 index during the live evaluation windows. I frame the task as a cross-sectional prediction problem: on each prediction date, the model assigns a score to every stock in the CSI500 universe based on historical price-volume information, and the portfolio is constructed from the highest-scoring names.

The competition evaluates performance by excess return relative to the CSI500 benchmark. Therefore, the model should not only predict whether a stock will have a positive return, but whether it is likely to outperform the index over the evaluation window. This motivated my main modeling choice: instead of predicting raw future stock returns, I predict future excess returns relative to the CSI500 index.

The final pipeline uses XGBoost regression models trained from scratch on historical market data. I use technical factors, liquidity factors, cross-sectional rank features, and an excess-return target. For the final submissions, I build four-model ensembles using both normal and conservative XGBoost models, and I adjust the forecast horizon weights according to the length of each live evaluation window.

2 Data and Task Setup

I use the provided CSI500 constituent list, daily stock OHLCV data, turnover, trading amount, and CSI500 index data. The stock close prices are forward-adjusted, which is appropriate for computing returns consistently across splits and dividends. The effective stock universe contains 499 stocks after filtering out constituents whose price data could not be downloaded successfully.

The portfolio constraints are:

- all stocks must be in the CSI500 constituent universe;
- the portfolio must contain at least 30 stocks with positive weights;
- the maximum weight of any single stock is 10%;
- weights must be non-negative and sum to 1 within the required tolerance.

The live evaluation metric is excess return:

$$e = R_p - R_b,$$

where R_p is the portfolio return and R_b is the CSI500 benchmark return over the same evaluation window. This scoring rule is important because it means that the model should focus on relative performance rather than absolute market direction.

3 Factors and Feature Engineering

The factor design is based on the idea that short-horizon stock selection can benefit from recent price trend, reversal, volatility, liquidity, and cross-sectional relative strength signals. All features are computed using only information available on or before the prediction date.

3.1 Momentum and Reversal Features

I include several recent return features:

$$\text{ret_1d}, \quad \text{ret_5d}, \quad \text{ret_10d}, \quad \text{ret_20d}, \quad \text{ret_60d}.$$

These features capture different horizons of recent price movement. Short-horizon returns can capture short-term reversal or continuation patterns, while longer-horizon returns can capture medium-term momentum. Since the competition evaluates portfolios over short windows, these recent return features are directly relevant.

3.2 Volatility Features

I use 20-day realized volatility:

$$\text{vol_20d}.$$

Volatility measures recent uncertainty and risk. It may also capture speculative trading activity. In a cross-sectional model, volatility can help distinguish stable relative strength from noisy price movements.

3.3 Liquidity and Trading Activity Features

I include:

$$\text{volume_z_20d}, \quad \text{turnover_ma_20d}.$$

The volume z-score measures whether a stock's current trading volume is unusually high relative to its own recent history. Abnormally high volume may indicate investor attention or information arrival. The turnover moving average captures recent trading activity and liquidity.

3.4 Trend and Technical Indicators

I include moving-average deviation and RSI features:

$$\text{close_over_ma20}, \quad \text{close_over_ma60}, \quad \text{rsi_14}.$$

The moving-average features measure whether the stock price is above or below its recent trend. RSI captures short-term overbought or oversold conditions. These are simple technical indicators, but they are useful in a tree-based model because XGBoost can learn nonlinear interactions between these features and future returns.

3.5 Cross-Sectional Rank Features

Because the task is cross-sectional stock selection, I also include daily rank-normalized features:

$$\text{ret_5d_rank}, \quad \text{ret_20d_rank}, \quad \text{vol_20d_rank}.$$

These features compare each stock to other CSI500 constituents on the same date. Rank features reduce sensitivity to raw scale differences across stocks and help the model focus on relative strength within the current universe.

4 Target Construction

The course baseline predicts raw forward stock returns. However, the competition evaluates excess return relative to the CSI500 index. Therefore, I changed the supervised learning target to forward excess return.

For a stock i on date t, the h-day forward stock return is

$$r_{i,t}^{(h)} = \frac{P_{i,t+h}}{P_{i,t}} - 1.$$

The corresponding CSI500 index return is

$$r_{m,t}^{(h)} = \frac{I_{t+h}}{I_t} - 1.$$

The target used by my model is

$$y_{i,t}^{(h)} = r_{i,t}^{(h)} - r_{m,t}^{(h)}.$$

This target directly matches the competition objective. A stock receives a high target value when it outperforms the CSI500 index over the future holding window.

I train both $h = 3$ and $h = 5$ models. Phase 1 uses a three-trading-day live window, so $h = 3$ is especially relevant. Phase 2 uses a five-trading-day live window, so $h = 5$ becomes more important.

5 Models

5.1 Normal XGBoost Model

The main model is an XGBoost regressor trained from scratch. XGBoost is appropriate for this task because the data is tabular, the features are a mix of continuous technical indicators and cross-sectional ranks, and the relationship between these factors and future excess return can be nonlinear.

The normal XGBoost model uses the following main parameters:

Parameter	Value
n_estimators	400
max_depth	5
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
min_child_weight	10
reg_lambda	1.0
early_stopping_rounds	30
random_state	42

Table 1: Normal XGBoost model parameters.

5.2 Conservative XGBoost Model

I also trained a more conservative XGBoost model. This version uses shallower trees, stronger regularization, a lower learning rate, and more conservative subsampling. The motivation is to reduce overfitting in a noisy short-horizon stock prediction problem.

Parameter	Value
n_estimators	800
max_depth	3
learning_rate	0.03
subsample	0.7
colsample_bytree	0.7
min_child_weight	20
reg_lambda	5.0
early_stopping_rounds	50
random_state	42

Table 2: Conservative XGBoost model parameters.

The conservative model was useful because it provided a slightly different ranking signal from the normal model. In Phase 1 testing, the conservative models achieved higher validation rank IC than the normal models, so I included both normal and conservative models in the final ensemble.

6 Training and Validation Methodology

I use time-based splitting rather than random splitting. Random train-test splitting would be inappropriate because stock returns are time dependent and would create look-ahead bias. For each training run, I restrict the training set to dates before the prediction date and use a validation window for early stopping and rank IC reporting.

I also apply an embargo between the training and validation windows. Since the target uses future returns, observations close to the validation period could have labels that overlap with validation features. The embargo reduces this leakage risk.

The main validation metric is daily cross-sectional rank IC, computed as the Spearman correlation between predicted scores and realized future excess returns on each validation date, then

averaged over validation dates. Rank IC is suitable because the portfolio is built from relative rankings of stocks rather than calibrated return forecasts.

7 Portfolio Construction

After training, the model produces a predicted score for each stock on the prediction date. The portfolio construction procedure is:

1. rank all eligible CSI500 stocks by predicted score;
2. select the top k stocks;
3. assign rank-based weights, with higher-ranked stocks receiving higher weights;
4. normalize weights to sum to 1;
5. enforce the 10% single-stock weight cap;
6. output a CSV with exactly two columns: `stock_code` and `weight`.

I use rank-based weights instead of raw score-based weights because raw model scores may not be well calibrated. Rank-based weights preserve the ordering information while avoiding excessive sensitivity to the numerical scale of XGBoost predictions.

For the final submissions, I use $k = 30$. This is the minimum allowed number of stocks, so it is more concentrated than a top-50 or top-80 portfolio. The self-test results showed that top-30 portfolios had the strongest average excess returns. In addition, Phase 1 live evaluation confirmed that the concentrated top-30 approach could perform well.

8 Self-Test Design

Before live submission, I designed a rolling historical self-test. For each historical as-of date, the model is trained using only data available before that date. The trained model predicts stock scores on the as-of date, constructs a portfolio, and evaluates the realized return over the next h trading days.

I tested two prediction horizons:

$$h = 3, \quad h = 5,$$

and three portfolio sizes:

$$k = 30, \quad 50, \quad 80.$$

The self-test covers 21 historical test windows. For each configuration, I report average excess return, median excess return, win rate, average portfolio return, and average benchmark return. This rolling setup is more realistic than a random split because it simulates the live competition setting.

9 Results

9.1 Baseline Comparison

The official baseline XGBoost model achieved a validation rank IC of 0.1794. After changing the prediction target from raw future stock return to future excess return, the validation IC improved

in earlier experiments. For example, the improved $h = 3$ and $h = 5$ excess-return models achieved validation rank IC values above 0.26 in the Phase 1 setup before the final ensemble construction.

The key improvement is conceptual: the supervised learning target is aligned with the live scoring metric. The model is trained to predict relative outperformance rather than raw return.

9.2 Rolling Self-Test Results

The 21-window self-test summary is shown below.

Horizon	Top-k	Avg Excess Return	Median Excess Return	Win Rate
3	30	0.010772	0.007389	0.857143
5	30	0.009476	0.007384	0.619048
3	50	0.007915	0.005222	0.809524
5	50	0.006158	0.007591	0.666667
3	80	0.005592	0.004848	0.714286
5	80	0.003871	0.002368	0.619048

Table 3: Rolling self-test results over 21 historical windows.

The strongest configuration in the self-test was $h = 3$, top-30. It achieved an average excess return of 1.0772%, a median excess return of 0.7389%, and a win rate of 85.71%. The top-30 portfolios outperformed the more diversified top-50 and top-80 portfolios in average excess return. This supported using a concentrated top-30 portfolio.

9.3 Phase 1 Live Result

For Phase 1, the live evaluation window had three trading days. Therefore, I used an $h = 3$-dominant four-model ensemble:

$$\text{score} = 0.35 \cdot z(\text{normal } h = 3) + 0.15 \cdot z(\text{normal } h = 5) + 0.35 \cdot z(\text{conservative } h = 3) + 0.15 \cdot z(\text{conservative } h = 5).$$

This gives 70% total weight to $h = 3$ signals and 30% total weight to $h = 5$ signals. The Phase 1 submitted portfolio achieved a realized portfolio return of

$$0.087653593 \approx 8.77\%.$$

It received the full live evaluation score. This result supported the overall modeling framework, especially the use of excess-return targets, ensemble models, and a concentrated top-30 portfolio.

9.4 Phase 2 Submission

Since the Phase 2 evaluation window has five trading days, I shifted the ensemble toward $h = 5$ models. I tested both a 70% $h = 5$-lead ensemble and an 80% $h = 5$-strong ensemble. The $h = 5$-strong version changed only 1 out of 30 names relative to the 70% $h = 5$-lead portfolio, so it was a small adjustment rather than a major strategy change.

The final Phase 2 score is:

$$\text{score} = 0.10 \cdot z(\text{normal } h = 3) + 0.40 \cdot z(\text{normal } h = 5) + 0.10 \cdot z(\text{conservative } h = 3) + 0.40 \cdot z(\text{conservative } h = 5).$$

This gives 20% total weight to $h = 3$ signals and 80% total weight to $h = 5$ signals. I selected the top 30 stocks by this ensemble score for the final Phase 2 submission.

10 Analysis: What Worked and What Did Not

10.1 What Worked

The most important improvement was changing the target from raw future stock return to future excess return relative to the CSI500 index. This directly aligned the supervised learning problem with the competition's live evaluation metric.

The second useful choice was using a four-model ensemble. The normal and conservative XGBoost models provide related but not identical ranking signals. Combining them reduced dependence on one specific hyperparameter setting. The conservative model was especially helpful because stronger regularization is appropriate for noisy short-horizon return prediction.

The third useful choice was using a concentrated top-30 portfolio. In the rolling self-test, top-30 portfolios had the highest average excess returns. The Phase 1 live result also supported this choice, since the top-30 ensemble portfolio achieved the full live evaluation score.

Finally, adjusting the ensemble horizon weights by phase was important. Phase 1 had a three-day evaluation window, so an $h = 3$-dominant ensemble was appropriate. Phase 2 has a five-day evaluation window, so I shifted the final submission toward $h = 5$ signals.

10.2 What Did Not Work or Was Risky

Adding more features is not automatically better. I tested a larger feature set earlier, but it reduced validation performance. One likely reason is that short-horizon stock returns are very noisy, and adding too many weak features can increase overfitting. Another reason is that long-window features reduce the number of usable training rows because they require more historical data.

Rank IC was also noisy across validation windows. This is expected in short-horizon stock prediction. For this reason, I did not rely on a single validation IC number. I used rolling self-tests and portfolio-level excess returns to evaluate whether a model configuration was robust.

A highly concentrated portfolio is also risky. Top-30 portfolios can produce stronger excess return if the ranking signal is correct, but they can also underperform if a few high-weight stocks move against the model. I accepted this risk because the grading metric is ranking-based and the self-test results favored top-30.

10.3 Limitations

The model only uses price-volume data and simple technical factors. It does not use fundamentals, industry classifications, analyst forecasts, or news data. It also does not explicitly enforce industry neutrality or market-cap neutrality. These could be useful extensions in a more realistic investment setting.

The self-test period is limited by the available data. Although I used 21 rolling windows, short-horizon portfolio returns remain noisy. Therefore, the live evaluation result can differ from historical self-test performance.

11 Reproducibility

All code used to generate the submissions is included in the code zip. The main scripts are:

- `my_features.py`
- `my_pipeline.py`
- `my_pipeline_conservative.py`
- `my_backtest.py`
- `make_4model_ensemble_submission.py`
- `make_week2_4model_ensemble_h5strong.py`

The final portfolio files are:

- `submissions/week1_SUBMIT.csv`
- `submissions/week2_SUBMIT.csv`

The self-test result files included are:

- `results/selftest_summary_21windows.csv`
- `results/selftest_results_21windows.csv`

The XGBoost random seed is fixed at `random_state = 42`. The README explains how to rerun the data update, model training, self-test, and final portfolio construction steps.

12 Academic Integrity and LLM Assistance

I completed this project individually. I used the course-provided baseline code as a starting point and modified the target construction, model configuration, ensemble construction, self-test procedure, and final portfolio generation.

I used ChatGPT as a coding and brainstorming assistant for debugging, pipeline organization, and report planning. The submitted portfolios were generated by my own trained XGBoost models using the provided market data. I did not use an LLM to directly choose stocks or generate portfolio weights.

13 Conclusion

This project shows that aligning the machine learning target with the evaluation metric is important in quantitative stock selection. Predicting forward excess return relative to the CSI500 index is more suitable than predicting raw stock return. The final approach combines technical and cross-sectional factors, normal and conservative XGBoost models, rolling self-tests, and phase-specific ensemble weighting.

The Phase 1 portfolio achieved a realized return of 8.77% and received the full live evaluation score. For Phase 2, I kept the same core pipeline but shifted the ensemble toward $h = 5$ signals to match the five-day evaluation window. Overall, the results support the use of excess-return targets, ensemble modeling, and concentrated top-30 portfolios for this competition setting.